

Statistics and Data Science for Engineers
Final Project Report
Team 35

INTRODUCTION AND PROBLEM DESCRIPTION

The semiconductor industry is currently seeing a massive increase in demand in the wake of the ongoing AI race. Thus, the manufacturing processes for the wafers on which all chips are built are in a state of constant improvement and automation. The semiconductor manufacturing process undergoes three main stages: lithography, etching, and deposition. The lithography process creates a template within the silicon wafer where the desired circuits will go. This is followed by the etching process, in which any unwanted material from the wafer is removed using chemicals or lasers. Finally, the deposition process adds any additional materials to the wafer, including conductors or insulators, preparing the wafer for electronics. These processes require an extreme level of precision and tightly controlled conditions in order to produce a successful wafer. Despite the substantial amount of improvement and research conducted in these manufacturing processes, creating a non-defective wafer still remains inconsistent.

To account for this inconsistency, several primarily image-based methods have been developed to identify defects within a semiconductor. These methods often use a defect's location, size, and type during manufacturing. While they have proven to be successful, they are limited by the time required to classify defects. As a result, there is an increasing demand for developing a quick and efficient model capable of real-time defect detection in an industrial production setting. Our group hopes to address this problem through the use of several machine learning algorithms, utilizing real-time sensor data obtained from the process stages of semiconductor manufacturing. Through Logistic Regression, Adaboost, Random Forest, and Support Vector Machines (SVM), we aim to create a successful model to categorize a dataset of semiconductors as defective versus non-defective.

DESCRIPTION OF THE DATA

The dataset we utilized is the “[Semiconductor Sensor Data for Predictive Quality](#)” from Kaggle. It contains a total of 4219 entries fitted on 10 numerical features. The features utilized are the Chamber Temperature, Gas Flow Rate, RF Power, Etch Depth, Rotation Speed, Vacuum Pressure, State Alignment Error, Vibration Level, UV Exposure Intensity, and Particle Count. The entries' output were split into the classes 'defects' and 'non-defects'. The data is shown to be highly imbalanced, however, containing 3602 entries classified as 'non-defects' as compared to 617 entries classified as 'defects'. This posed a challenge in processing the data, as many of our models had a recurring issue where the models would automatically classify all the test entries as non-defective. This provided an extra challenge in processing our data, resulting in our team looking outward in determining what methods we use in order to classify our data. One method we utilized was splitting the majority chunk, or the data classified as 'non-defects', into several smaller chunks, resulting in several smaller groups of data, with a better output ratio.

This eliminated the error where the data would assume the wafer is non-defective. However, this also results in the training data being much smaller in size and not encompassing the entire dataset, resulting in a less comprehensive model.

METHODOLOGY

Note: In addition to the below four datasets, we attempted both gradient boosting and neural networks. However, neither of these methods produced notable results, so we have intentionally omitted them from this report and the final submission. Furthermore, we have tested metrics such as MCC, on top of the other metrics already mentioned in class, in an attempt to better classify our data.

3.0 Evaluation Metric

To evaluate our models, we chose the FBeta (Beta = 2) score because catching defective chips matters far more than accidentally flagging good ones, so recall needs to be weighted more heavily. While recall minimizes our rate of shipping false negatives, many of the models would maximize recall by classifying all chips as defective. This ultimately led us to evaluate based on the F2 score. Moreover, we also noticed that some models ‘cheated’ by predicting almost everything as defective, which boosted recall and F2 without actually doing anything useful. Since our dataset is pretty imbalanced, we added MCC to get a clearer picture of whether a model was actually learning meaningful patterns rather than taking advantage of the class distribution

3.1 Data Cleaning and Preprocessing

As we see from the above description of the data, we had columns such as Process ID, Timestamp, and Wafer ID, which did not play a part in the prediction, and so the first step was to drop those columns. Moreover, the ‘Defect’ column, which mentions ‘0’ for non-defective wafers and ‘1’ for defective, is linked directly to the last column, which categorizes the wafers as joining/non-joining. Hence, we dropped this column as well. The tool type column categorizes each of the data points based on the process they were recorded in and is categorical. So, we one-hot encoded this and set it such that as soon as it detects ‘1’ for any of the three types, it drops the rest, as a wafer could only belong to one of the three and not doing so would lead to unnecessary correlations being established between them (multicollinearity). All of these steps are done in Section 2.1 of the notebook. In Section 2.2, we create an 80-20 split of the data into training and testing sets, respectively, and subsequently use the Standard Scaler for normalizing all the features so that those with bigger absolute values do not gain numerical weightage over the smaller ones.

3.2 Logistic Regression

Naturally, we started with logistic regression, given that it's the primary binary classification model. Alongside optimizing the hyperparameters -inverse regularization strength (C) and regression type (Lasso vs Ridge) - we also attempted to improve the performance by undersampling and tuning the prediction probability thresholds. Before moving on to the baseline modelling, we have visualized the feature importance using L1- regularization penalty, which tends to shrink unimportant feature coefficients to exactly zero (see Figure 1 below).

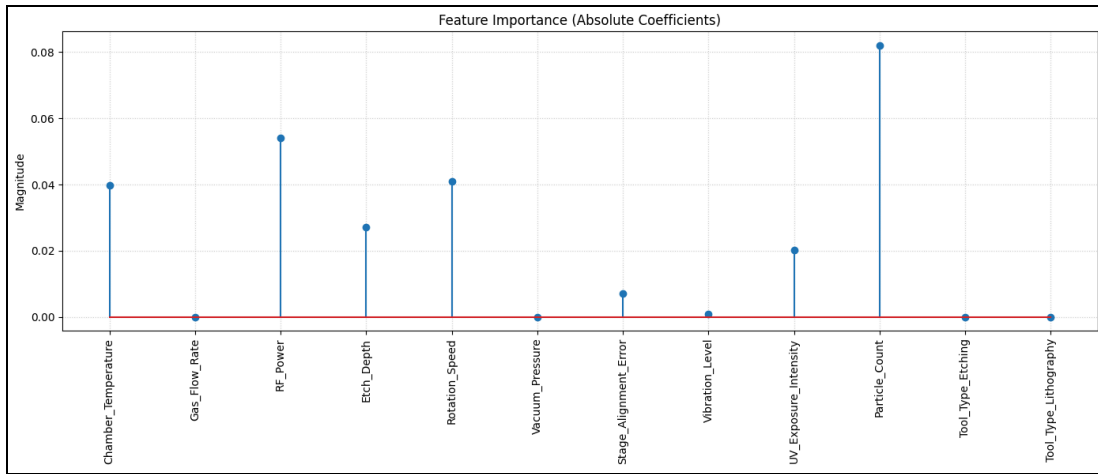


Figure 1: Feature importances using L1-regularization

3.2.1 Baseline Model

In Section 2.3.2, we train a basic logistic regression model with balanced class weights (due to class imbalance) and 5000 maximum iterations. We train a basic logistic regression model with balanced class weights (given the imbalanced nature of our data), and 5000 max iterations. Furthermore, we performed stratified K-Fold cross-validation to ensure representative testing across all classes and utilized the F2 score to prioritize recall, explicitly evaluating the model's ability to minimize false negatives rather than simply maximizing accuracy. Through this model, we get the following values to base our further model performances on -

F2: 0.33, Recall: 0.47, Precision: 0.156

3.2.2 Hyperparameter Tuning

Our tunable hyperparameters are the penalty method (L1 or L2) and the inverse regularization strength (C). We perform a manual grid search over these parameters, validating each combination via stratified cross-validation to identify the specific configuration that optimizes model complexity and maximizes the F2 score on the minority class.

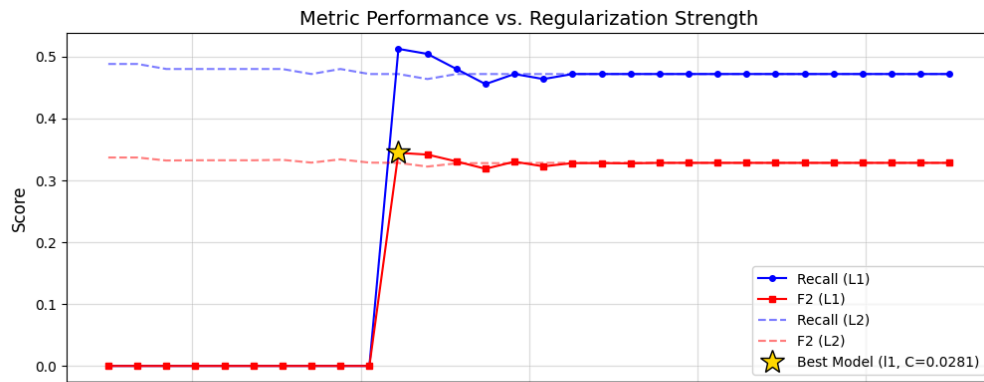


Figure 2. Metric Performance vs Regularization Strength

We find that the best inverse regularization strength for the above is $C=0.0281$ with the penalty of 'L1' or Lasso Regularization. It yields the following values -

Recall = 0.5122, F2 = 0.3443, Accuracy = 0.5024, and Precision = 0.1489 and is thus only a marginal improvement over the baseline model.

3.2.3 Further Attempts to Improve Performance

3.2.3.1 Threshold Tuning (Section 4.3)

This involves adjusting the decision boundary for classification beyond the default 0.5. For probabilistic classifiers, a sample is predicted as defective if $P(\text{Defect}|X) \geq \text{threshold}$.

We evaluated two approaches to this end - (1) grid search over thresholds [0.1, 0.9] optimizing balanced accuracy, which is a mean of the precision and recall scores (in Section 4.3.1), and (2) G-mean optimization using the ROC curve (4.3.2). The G-mean method selects the threshold that maximizes G-mean, balancing the true positive and true negative rates. Lower thresholds increase recall (catching more defects) at the cost of precision (more false alarms). It yielded the following results as listed in Table 2.

$$G_{mean} = \sqrt{(TPR * (1 - FPR))} = \sqrt{(Sensitivity * Specificity)}$$

Thus, while the F2 score was slightly decreased, the overall model performance was better due to an improvement in both accuracy and precision, even though it came at the cost of recall.

We also investigated one other scoring method for threshold determination called Youden's J-Score. Youden's J statistic selects the threshold that maximizes $J = TPR - FPR$ (equivalently, $Sensitivity + Specificity - 1$), finding the point on the ROC curve farthest from the random classifier diagonal. While G-mean uses the geometric mean $\sqrt{(TPR \times (1-FPR))}$, Youden's J uses the arithmetic difference, typically yielding more aggressive thresholds that favor recall.

	G - mean	Youden's J-Score
F2	.3253	.4324
Recall	.4390	.7642
Precision	.1597	.1580
Accuracy	.5818	.3720

Table 1. Logistic Regression Evaluation

3.2.3.2 Undersampling (Section 4.4)

This was done to address the class imbalance by randomly sampling the majority class (Pass) to match the minority class (Defect) size, reducing training data from ~2880/494 to multiple chunks 494/494. The model is then trained on this balanced subset using the best hyperparameters (C) found from the original tuning as well as the threshold. This trades off information loss from discarded majority samples for equal class representation during training.

3.2.4 Final Model Comparison (Section 4.5)

To determine the best of all these closely performing models, we used the Matthews Correlation Coefficient (MCC), which measures the correlation between predicted and actual classifications.

It is calculated as the following, incorporating all four confusion matrix quadrants:

$$MCC = \frac{(True\ Pos) * (True\ Neg) - (False\ Pos) * (False\ Neg)}{\sqrt{(True\ Pos + False\ Pos)(True\ Pos + False\ Neg)(True\ Neg + False\ Pos)(True\ Neg + False\ Neg)}}$$

MCC is particularly useful here because a model that predicts all samples as "Defect" would achieve high recall but $MCC \approx 0$, exposing the lack of true predictive power.

	MCC	Recall	F2	Accuracy	Precision
Vanilla (C=1.0, threshold (t)=0.5)	0.0039	0.4715	0.3273	0.5249	0.1472
Best Penalty = 'L1' & C=0.0281, t=0.5	0.0091	0.5122	0.3443	0.5024	0.1489
Best C + tuned threshold (G_Mean)	0.0325	0.4390	0.3253	0.5818	0.1598
Best C + tuned threshold (J Score)	0.0537	0.7642	0.4324	0.3720	0.1580
Undersampled + tuned threshold (G Mean), C = 0.08857	0.0643	0.6341	0.4054	0.4822	0.1660

Table 2. Logistic Regression Final Model Comparison

As we can see from the above table, the best performing model for us was the undersampled version, which had the tuned threshold, L-1 regularization, as well as the best undersampled inverse regularization strength of 0.08857.

However, all in all, the performance of the model is still not much better than random guesses, which prompted us to explore other models.

3.3 Support Vector Machines

3.3.1 Baseline Model

For the baseline SVM model, we used a linear kernel with balanced class weights to counter the class imbalance. As expected, without any optimization, the model struggled to separate defective from non-defective wafers in any meaningful way. The performance was only slightly better than random guessing, with the following metrics: F2 = 0.35, Recall = 0.53, Precision =

0.15, and Accuracy = 0.50. This made it clear that a straightforward linear SVM would not be sufficient unless we adjusted its regularization behavior or decision boundaries.

3.3.2 Hyperparameter Tuning

We then explored a range of hyperparameters to push the model beyond this baseline. Specifically, we tuned the regularization strength (C) over a logarithmic range from 10^{-2} to 10^2 , experimented with kernel boundary flexibility (γ) across the same scale, and compared three kernel types - linear, RBF, and polynomial. Across these experiments, the linear kernel consistently outperformed the RBF and polynomial kernels, likely because our feature space (after scaling) did not benefit from additional nonlinearity and because the noise in the data made more flexible kernels prone to overfitting.

To address the imbalance and improve generalization, we also applied the undersampling techniques used in the earlier models. We trained multiple linear SVMs on different undersampled subsets and averaged their predicted probabilities. The idea was to reduce the model's exposure to the overwhelmingly non-defective class while stabilizing its predictions through ensembling.

3.3.3 Model Evaluation

The best-performing configuration turned out to be an undersampled linear SVM with a C value of 0.007, which produced: $F2 = 0.38$, Recall = 0.60, Precision = 0.16, and Accuracy = 0.47. While the boost in recall was encouraging, the precision remained low, which ultimately limited the improvement in the $F2$ score. The very low optimal C value also suggests that the model needed heavy regularization to avoid overfitting the noisy feature space, further reinforcing the challenge of identifying meaningful structure in this dataset. While recall did improve over the base model, precision was still abysmal, resulting in an $F2$ score of .38. This, combined with a very low C score, once again demonstrated the difficulty the models had cutting through the noise to identify important features.



Figure 3. SVM Confusion Matrices

3.4 Random Forest

Random forest was chosen as one of our models as it is well-suited for tabular data, which matches the structure of the dataset. Random forest handles class imbalance through class weighting, which is important given the imbalance in the dataset.

3.4.1 Baseline Model

For the baseline model, a vanilla random forest was used with a simple config of 10 trees with no limits on tree depths. There was also class balancing enabled to account for the imbalanced dataset. Several key metrics were evaluated, including MCC, F2, recall, precision, and accuracy to determine a reference for hyperparameter tuning. The results are as follows:

F2: 0.0101, Recall: 0.0081, Precision: 0.2500, Accuracy: 0.8519

3.4.2 Hyperparameter Tuning

To optimize random forest, two hyperparameters were tested. These were the number of estimators (`n_estimators`) and max tree depth (`max_depth`). Six values for `n_estimators` were used (1-100) and four values for `max_depth` (1-15) were evaluated with class balancing enabled. Performance was assessed using cross-validation and metrics including MCC, F2, and recall. The results (Figure 4) show that recall and F2 decreased as trees became deeper and ensembles larger. This shows that simpler configurations were more effective for identifying the minority class. The MCC varied across settings without a clear trend which shows that the overall predictive balance was sensitive to hyperparameter choices.

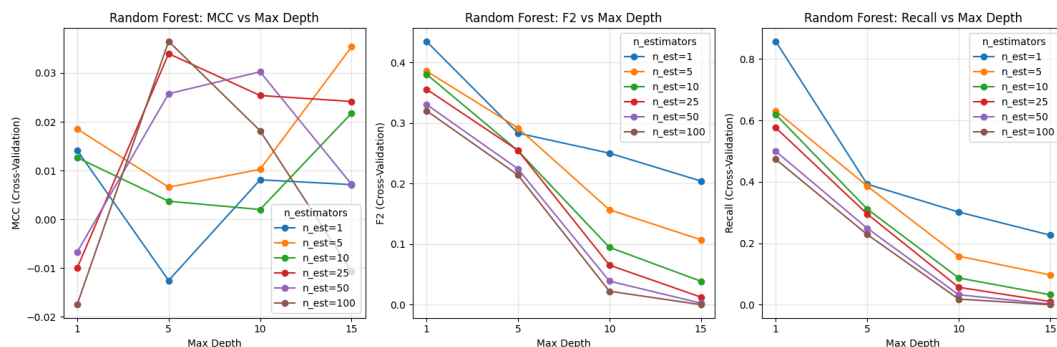


Figure 4. Random Forest Performance Metrics vs Max Depth for Different Numbers of Estimators

3.4.3 Further Attempts to Improve Performance

3.4.3.1 Threshold Tuning

Threshold tuning was applied to the best random forest model to improve its performance. Using both G-mean and Youden's J methods, the optimal threshold was 0.2689 for converting predicted probabilities into class labels. With this threshold, the model achieved an MCC of 0.0515, F2 score of 0.3753, recall of 0.5528, precision of 0.1643, and an accuracy of 0.5249. These results showed that adjusting the decision threshold can increase the model's performance, though the overall predictive performance remains limited by the dataset imbalance.

3.4.3.2 Undersampling

To address class imbalance, the majority class was split into six chunks and combined with all minority samples to create balanced training subsets. A random forest was trained on each subset, and predictions were averaged to produce final probabilities. With a 0.5 threshold, the ensemble achieved an MCC of 0.0338, F2 of 0.3706, and a recall of 0.5610. The confusion matrix showed improved detection of defects, but the accuracy was reduced since there were fewer majority class samples.

3.4.4 Final Model Comparison

To compare the different random forest models, each configuration was evaluated using multiple metrics, which include MCC, recall, F2, accuracy, and precision. Five variants were considered: vanilla, best-tuned hyperparameters, threshold tuning using G-mean and Youden's J, and the undersampled ensemble. Figure 5 summarizes the test set performance for each.

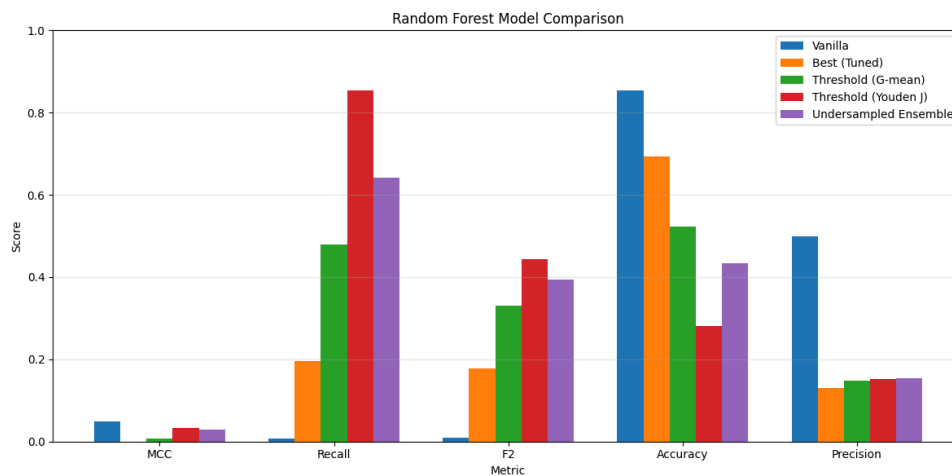


Figure 5. Random Forest Model Comparison

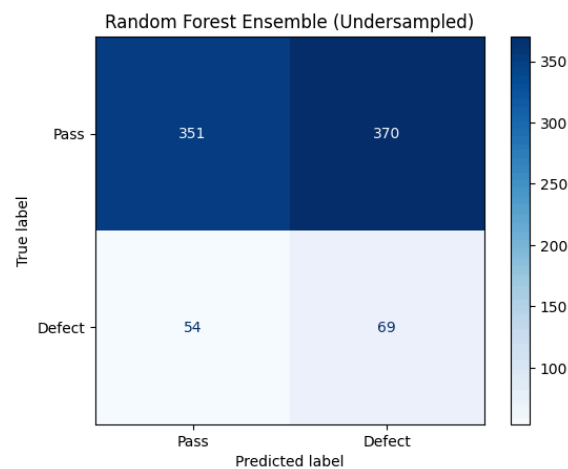


Figure 6. Random Forest Best Confusion Matrix

Here in Figure 6, the matrix shows that the model correctly predicted 351 instances of the majority class and 69 instances of the minority class, while misclassifying 370 majority class instances and 54 minority class instances.

As shown, the baseline random forest achieved a very low MCC (0.0039) with a moderate recall (0.4715), and F2 of (0.3273). Hyperparameter tuning improved F2 slightly, but the threshold tuning notably increased recall and F2, with Youden's J threshold producing the highest recall (0.5528) among the single-model approaches. The undersampled ensemble achieved the highest recall (0.5610) and comparable F2 (0.3706). This shows that balancing the classes during training helps detect the minority class. However, the MCC remained low across all models, which shows that the overall predictive power is limited due to the class imbalance.

3.5 AdaBoost Classifier

The AdaBoost model was included as one of the ensemble learning approaches. Since the dataset was heavily imbalanced, AdaBoost models on regular training datasets could not come up with proper recall. Therefore, further data preprocessing was required to account for the heavy class imbalance.

3.5.1 Motivation

The defective wafers represent a very small proportion of the total dataset. Since missing a defective wafer is significantly more costly than incorrectly flagging a non-defective one, the main objective is to maximize recall on the defective class.

3.5.2 Undersampling for Class Imbalance

The dataset is highly imbalanced, so a special downsampling strategy was applied before training the AdaBoost models. The non-defective samples were manually divided into multiple equal-sized chunks to create 6 balanced datasets, each paired with the same minority set of defective wafers.

- Total non-defective (majority) samples: 3602
- Total defective (minority) samples: 617
- Number of majority chunks created: 6
- Sizes of majority chunks: [494, 494, 494, 494, 494, 411]
- Size of minority set included in every balanced dataset: 494

Each chunk was concatenated with the minority subset, shuffled and used to train one AdaBoost model. This created an ensemble of models trained on different balanced views of the data, and also prevents the problem of loss of training data.

3.5.3 Model Configuration

Each balanced dataset was used to train a separate AdaBoost classifier. AdaBoost hyperparameters explored:

- Learning rates: 0.1, 0.3, 0.5, 1.0

- Number of estimators: 10, 25, 50, 100, 200, 400

This grid was selected to try out at least 10 variations of hyperparameters for hyperparameter tuning.

Since six AdaBoost models were trained for each combination of hyperparameters, the predictions were combined at the probability level. For each test sample, the defective-class probabilities from all models were averaged. The final prediction was made by comparing this averaged probability to a chosen classification threshold, which is an essential step in converting these probabilities into final class labels. The classification threshold by default is set to 0.5. That is, samples with predicted probability ≥ 0.5 are labelled as defective, and those below 0.5 are labelled as non-defective.

4.5.4 AdaBoost Performance

All experiments were evaluated using recall on the defective class, since identifying defective samples is the primary objective of this project. A grid of learning rates and numbers of estimators was tested.

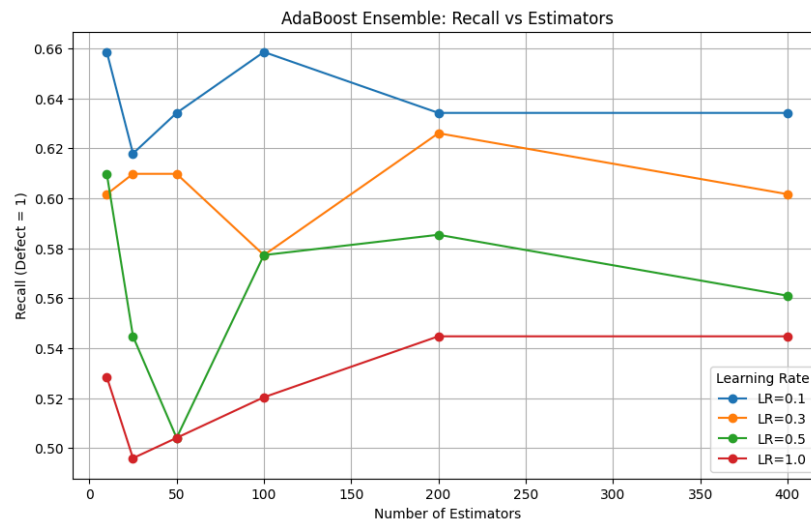


Figure 67 Adaboost Ensemble: Recall vs Estimators

- Learning Rate = 0.1, n_estimators = 10
- Learning Rate = 0.1, n_estimators = 100

The recall values remain within the approximate range of 0.50 to 0.66, indicating that AdaBoost is only able to recover roughly half to two-thirds of the defective samples. This trend was consistent across all learning rates and estimator counts, suggesting that:

1. AdaBoost with decision stumps struggles to separate the defective class from the non-defective class under this data distribution.
2. The downsampled-chunk ensemble helps stabilize performance but cannot push the model beyond this recall plateau.

- Increasing the number of estimators does not meaningfully improve performance, which indicates that the model is limited by the underlying weak learner and class imbalance structure.

Results and Analysis

As shown in Table 4 Final Results, our best model (as judged by the F2 score) is Adaboost. Additionally, Adaboost has a recall score of .72, critically without maximizing recall by classifying every wafer as defective. Additionally, Figure 8 Shows the confusion matrix comparison for each of our models.

Model	Hyperparameters	F2	Recall
AdaBoost	Learning Rate = .1 Estimators = 100	0.41	0.72
Logistic Regression	Penalty = 11 C = 0.0281 t = 0.5	0.41	0.63
Random Forest	N_estimators = 5 Max_depth = 15 Class_weight = balanced	0.37	0.56
SVM	C = .007 Kernel = linear	0.36	0.54

Table 3. Final Results

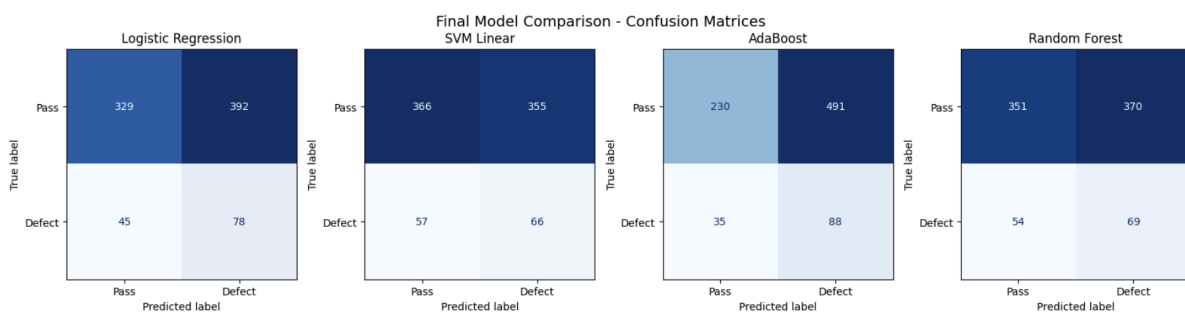


Figure 8. Final Model Confusion Matrix Comparison

However, it should also be noted that all of these models perform poorly. Our best F2 score was .41, and all models performed poorly on precision during testing. This is likely due to our dramatically imbalanced dataset, and could be rectified by adding more examples of defective data to pull from. Ultimately, with the data available, we would say it is not possible to create effective statistical models of this manufacturing process.